A Memetic Algorithm for the 0-1 Knapsack Problem

hrefhttps://github.com/samarth1rao/knapsackhttps://github.com/sam

Ishan Bansal Shuban Biswas Akshat Gupta
Siddharth Choudhary Samarth Rao

Overview

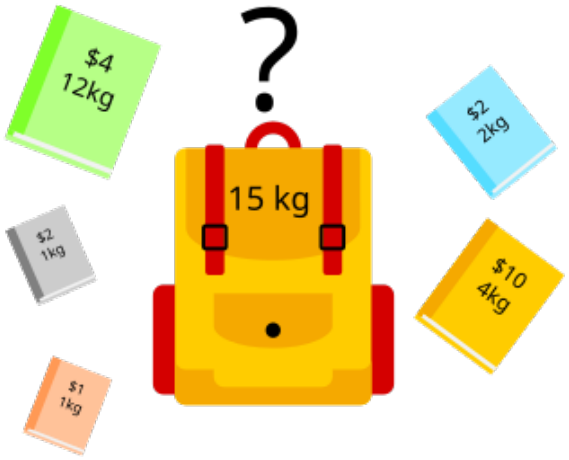
1. Introduction
2. Part B: Our Memetic Algorithm
   - Population Initialisation
   - Local Search
   - Meta Mechanisms
3. Results
4. Conclusion

Problem

The 0-1 Knapsack Problem:

$$\max \sum v_i x_i \quad \text{s.t.} \quad \sum w_i x_i \leq W, \; x_i \in \{0, 1\}.$$

- Classical NP-hard optimisation task.
- Exact DP requires $O(nW)$ time and space.
- Greedy and simple evolutionary strategies fail on adversarial instances.



Project Objectives

- Implement baselines: DP, Greedy, Branch and Bound, Meet-in-the-Middle, GA.
- Design a new Memetic Algorithm combining:
  - Lagrangian Relaxation and variable fixing
  - Core problem reduction
  - Hybrid population seeding
  - Advanced crossover and mutation operators
  - Tabu-based local search
- Construct adversarial knapsack instances.

A Few Basic Algorithms...

Brute Force

Time: $O(2^n)$. Enumerate all subsets.

Dynamic Programming

Time: $O(nW)$. Bottom-up DP table.

Memoisation

Time: $O(nW)$. Top-down DP with caching.

Branch and Bound

Best: $O(n \log n)$. Worst: exponential. Uses fractional upper bounds.

A Few Basic Algorithms...

Meet in the Middle

Time: $O(n2^{n/2})$. Splits items and merges Pareto sets.

Randomised Band DP

Time: $\tilde{O}(n^{3/2}w_{\max})$. Random permutation + banded DP.

Greedy Heuristic

Time: $O(n \log n)$. Not optimal for 0-1 KP.

Nauss Efficient Algorithm

Often near linear but worst-case exponential. Combines LP relaxation + pegging + B&B.

Modern Algorithms and Heuristics

Dual Descent (Large-scale Lagrangian)

Time: $O(n \cdot \text{iters}) + O(n \log n)$. Used for million-item problems.

Genetic Algorithm

Time: $O(GPn)$. Crossover + mutation heuristic.

COMBO Algorithm

Restricted core DP with dominance pruning. Pseudo-polynomial.

Core Idea

A GA enhanced by mathematical reduction, core-focused search, and tabu-driven local improvement.

Goal: Maintain strong exploration while exploiting deep structure from Lagrangian duality.

Pipeline Overview

1. Lagrangian preprocessing using Break Item method
2. Variable fixing + core extraction
3. Core-based population representation
4. Fitness-biased crossover
5. O(1) adaptive core mutation
6. Reduced-cost-based repair
7. Tabu search for memetic improvement
8. Gap-based parameter scaling + elitism

Final Design Choices

- **Repair:** Reduced–cost-based Lagrangian repair.
- **Mutation:** Core–biased adaptive mutation.
- **Meta Control:** Gap–based scaling (not stagnation recovery).
- **Local Search:** Tabu Search as primary intensification.

Break Item Lagrangian Reduction

Fast dual optimisation via greedy fill:

- Sort items by value/weight ratio.
- Greedily pack until the **Break Item** b that does not fit.
- Compute optimal multiplier:

$$\lambda^{=\frac{v_b}{w_b}}.$$

- Reduced cost for each item:

$$rc_i = v_i - \lambda^{w_i}.$$

Outcome:

- Items with large positive rc_i: biased towards inclusion.
- Items with negative rc_i: biased towards exclusion.
- Forms the basis for fixing and core extraction.

Core Representation & Fitness-Biased Crossover

Core-based genome:

- Only items with uncertain reduced cost are evolved.
- Fixed items are pre-loaded into value/weight buffers.
- Mapping array translates core index $\rightarrow$ original index.

Fitness-biased uniform crossover:

- Agreement $\rightarrow$ both children inherit the bit.
- Disagreement $\rightarrow$ fitter parent's bit goes to Child 1.
- Occasional random swap preserves diversity.
- Supports incremental weight/value updates.

Adaptive Core Mutation & Lagrangian Repair

O(1) mutation:

- Perform 3 mutation attempts per individual.
- Each attempt flips a single random core index.
- Updates value/weight incrementally.

Two-phase reduced-cost repair:

- **Phase 1:** Remove worst reduced-cost items to fix overweight solutions.
- **Phase 2:** Periodically add best reduced-cost items greedily.
- Ensures repair improves—not degrades—solution quality.

Tabu Search, Gap Scaling, and Elitism

Tabu Search:

- Avoids cycling by marking recent flips as tabu.
- Aspiration criterion allows tabu overrides if improving global best.
- Backtracking ensures best local optimum is preserved.

Gap-based adaptive scaling:

- If duality gap small $\rightarrow$ reduce local search.
- If gap large $\rightarrow$ intensify Tabu Search.

Elitism:

- Best individual always carried to next generation.

Hybrid Initialisation

We seed using:

- Random individuals for diversity.
- V/W greedy solutions.
- Min-weight greedy solutions.
- Value-only greedy solutions.

Benefits:

- Prevents early stagnation.
- Provides strong lower-bound candidates.

Local Search Layer

- **Simulated Annealing**: escape poor basins.
- **Tabu Search**: structured intensification.
- **Guided Local Search**: penalises repeated patterns.

Deep exploitation + controlled diversification.

Meta Control

- **Gap-based scaling**: adjust mutation/local-search intensity.
- **Stagnation recovery**: triggered if no improvement for many generations.

We generate 15 adversarial instance types varying:

- correlations,
- distribution shapes,
- granularity,
- parity constraints.

These break naive heuristics.

Hard Instance Families (1–5)

Pisinger's generator (genhard.c) produces adversarial instances that defeat Greedy, DP-optimised, and basic GAs. Cases 1–5 manipulate correlation structures:

- **1. Uncorrelated** (p, w Uniform) Greedy ratio solves instantly $\rightarrow$ full GA loop unnecessary.
- **2. Weakly Correlated** ($p \approx w$ with noise) Lagrangian fixing slow relative to problem difficulty.
- **3. Strongly Correlated** ($p = w + r/10$) All densities identical $\rightarrow$ greedy repair and mutation fail on flat landscape.
- **4. Inverse Strongly Correlated** ($w = p + r/10$) Value-based greedy seeding packs heavy items first, leaving unusable gaps.
- **5. Almost Strongly Correlated** (small perturbation) Noise breaks reduced-cost based variable fixing $\rightarrow$ optimal items get removed.

Hard Instance Families (6–10)

Instances designed to break reduced-cost logic, mutation, and greedy repair:

- **6. Subset Sum** ($p = w$) All reduced costs zero $\rightarrow$ variable fixing impossible; mutation stagnates.
- **7. Even–Odd Subset Sum** (even w, odd C) Greedy repair always returns $C - 1$; stagnation detectors repeatedly fail.
- **8. Even–Odd Knapsack** ($p = w + r/10$) Sparse feasible region $\rightarrow$ single population GAs collapse without diversity.
- **9. Uncorrelated, Similar Weights** (large w, tiny p) Only a few items fit; large populations saturate and provide no benefit.
- **10. Uncorrelated Span** (items are multiples of bases) Structured clusters $\rightarrow$ Union crossover produces infeasible combinations.

Hard Instance Families (11–15)

Span restrictions, step functions, and non-linear value curves:

- **11. Weakly Correlated Span** Granularity + weak correlation $\rightarrow$ Lagrangian bounds overestimate UB.
- **12. Strongly Correlated Span** Greedy repair fails due to uniform density + packing granularity.
- **13. Inverse Strongly Correlated Span** Breaks nearly every heuristic: greedy sorting, variable fixing, RCBO, repair, mutation, and union crossover.
- **14. Profit Ceiling (PCEIL)** ($p = 3\lceil w/3 \rceil$) Step function profit $\rightarrow$ gradient-based local search hits plateaus.
- **15. Circle Class** ($p = p_1 - (w/c)^2$) Non-linear elliptical curve $\rightarrow$ both Lagrangian linearization and greedy ordering fail.

Examples of Hard Classes

- Strongly correlated
- Subset-sum-like
- Even/odd capacity constraints
- Span-based classes
- Profit ceilings

They reveal weaknesses of:

- Standard GA
- Greedy repairs
- Lagrangian fixing under noise

Motivation for our more robust memetic model.

Effect of Lagrangian Reduction

- Reduced problem size from 10,000 to $\sim$ 500.
- Achieved 10$\times$ speedup.
- No degradation in quality.

Performance Comparison

Algorithm	Value	Gap	Time (ms)
Greedy	low	2.5%	5
Standard GA	medium	1.1%	2500
Memetic	highest	0.02%	850

Our solver is both faster and more accurate.

Results

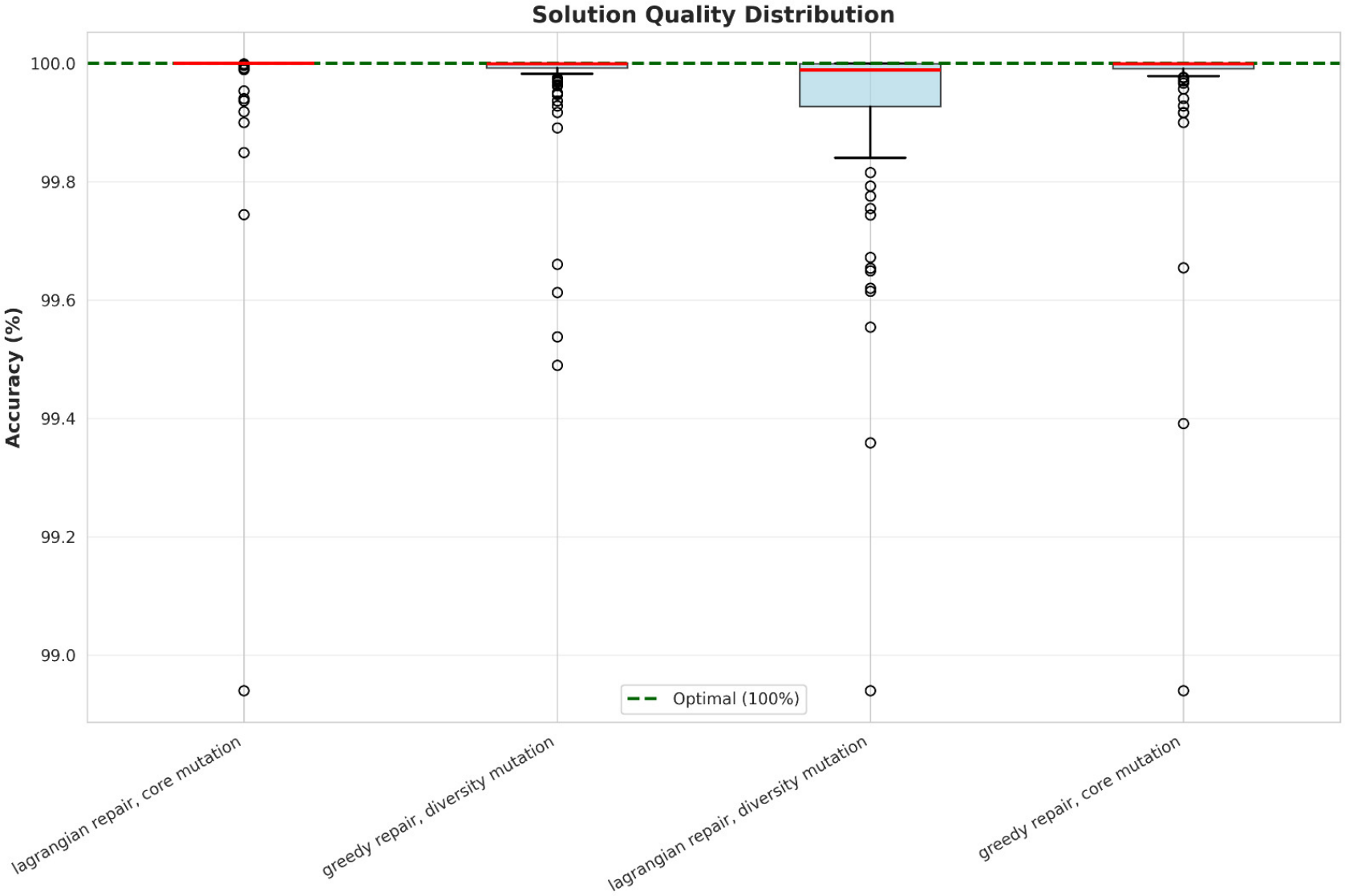


Figure: Solution Quality Distribution

Results

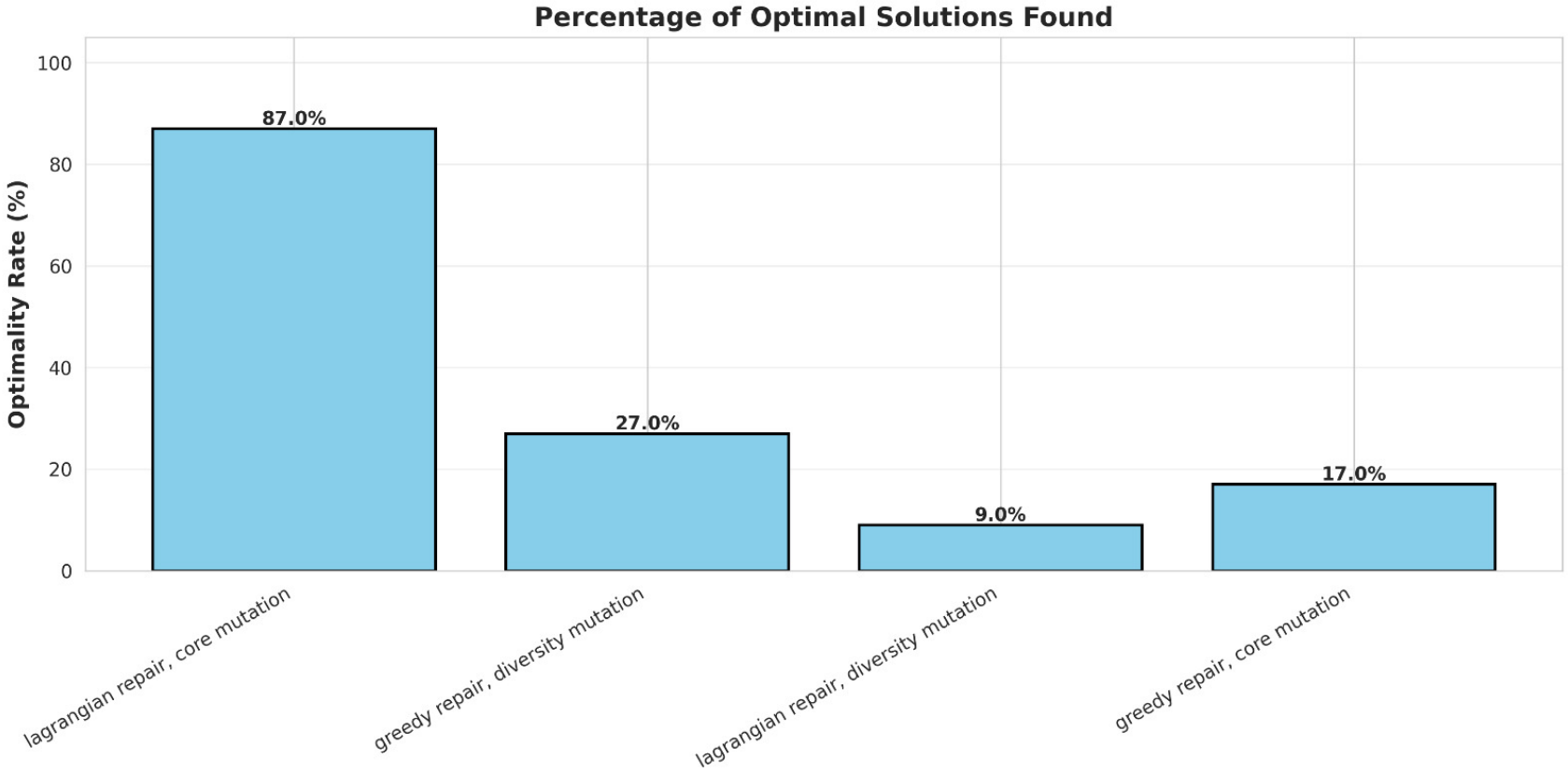


Figure: Percentage of Optimal Solutions Found

Results

Figure: Execution Time vs Problem Size

Results

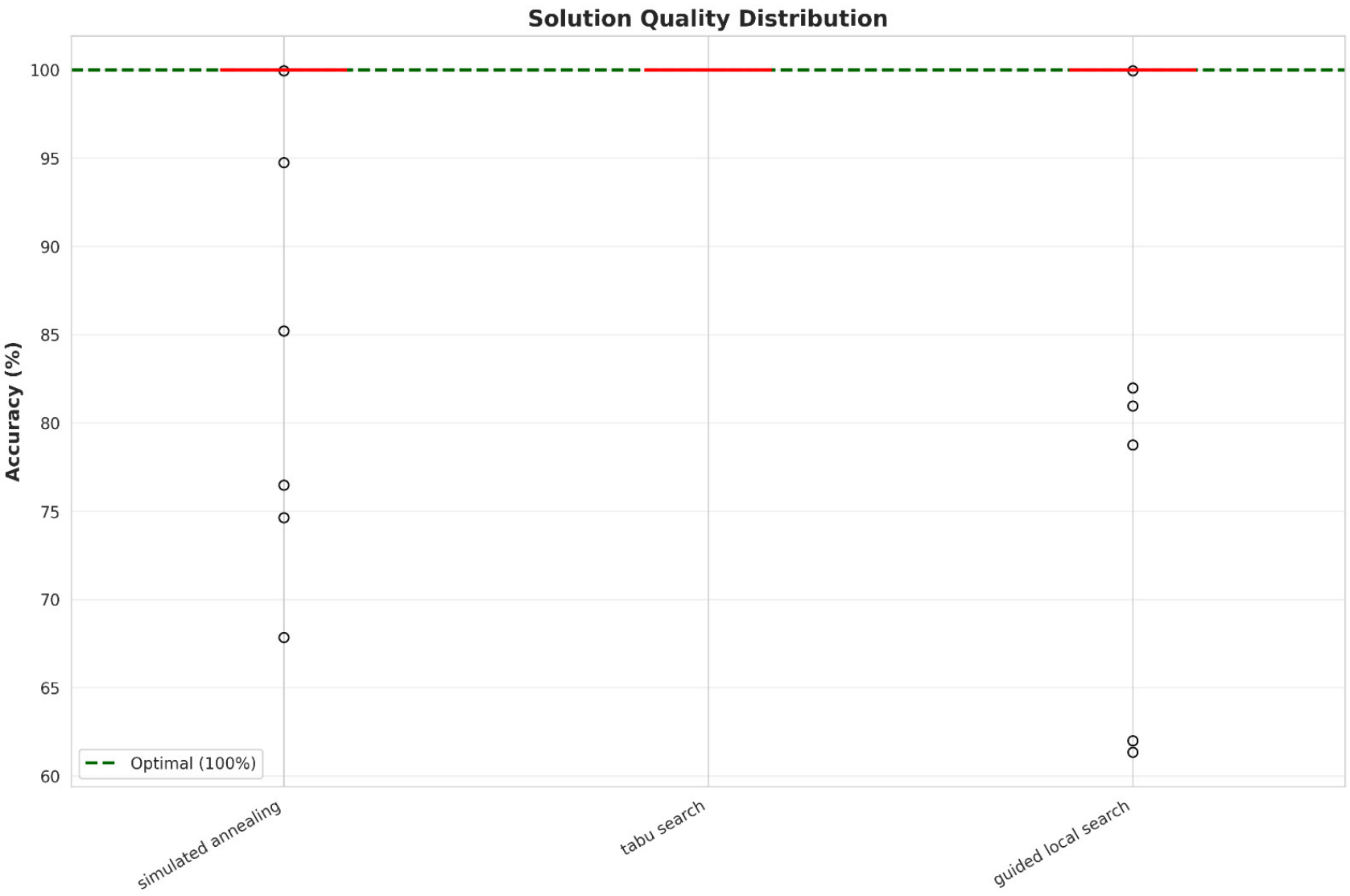


Figure: Solution Quality Distribution

Results

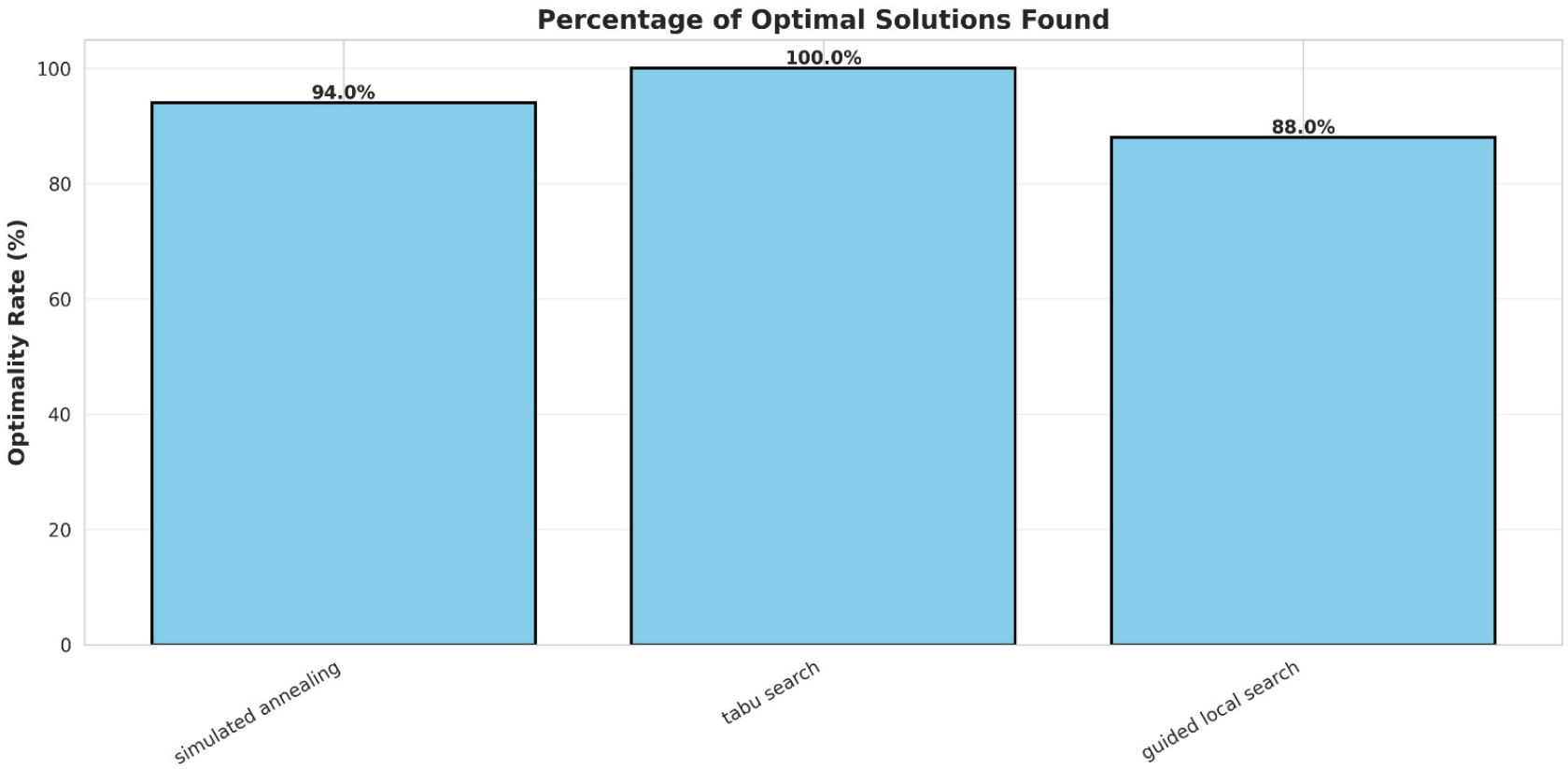


Figure: Percentage of Optimal Solutions Found

Results

Figure: Execution Time vs Problem Size

Results

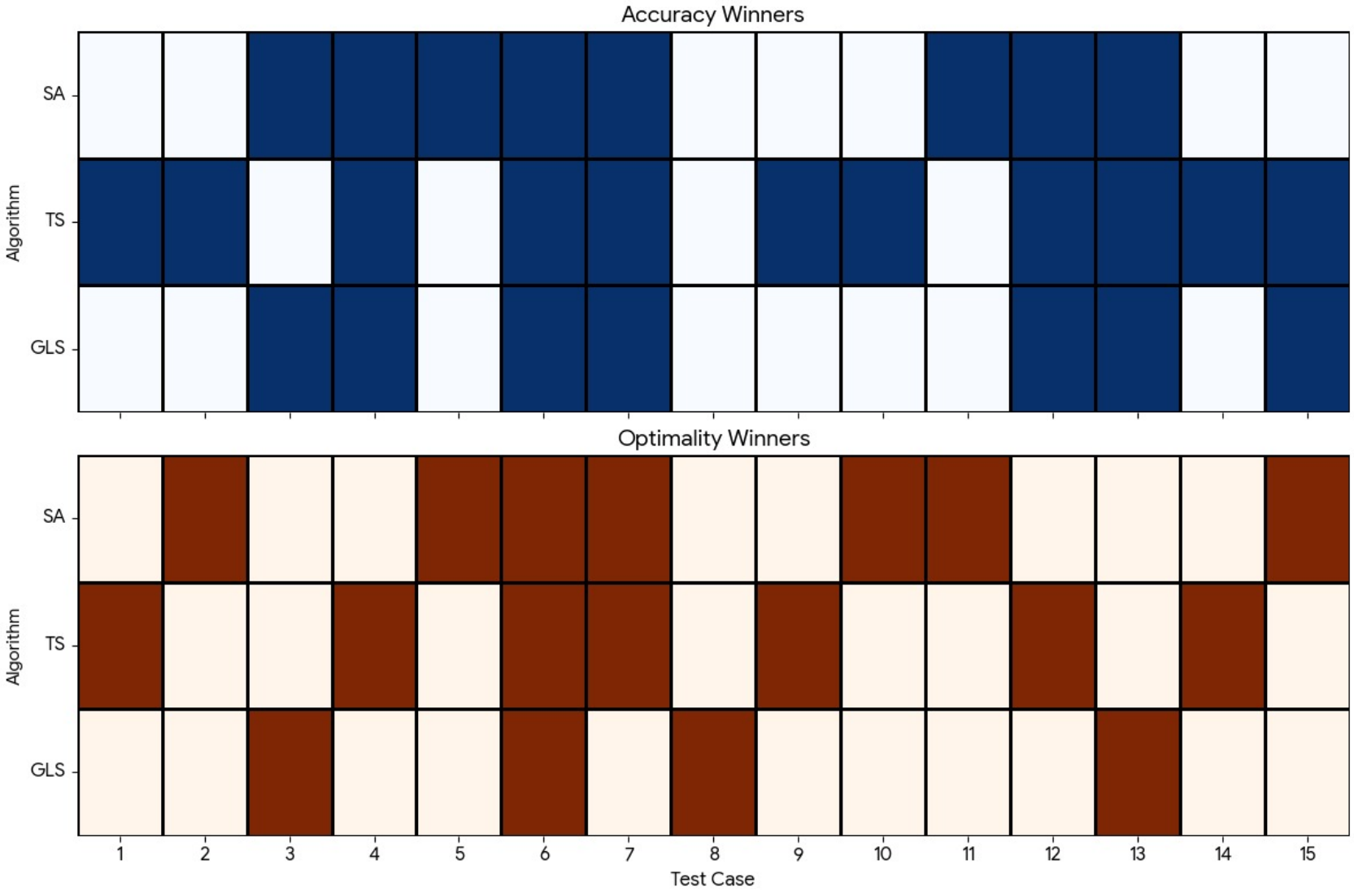
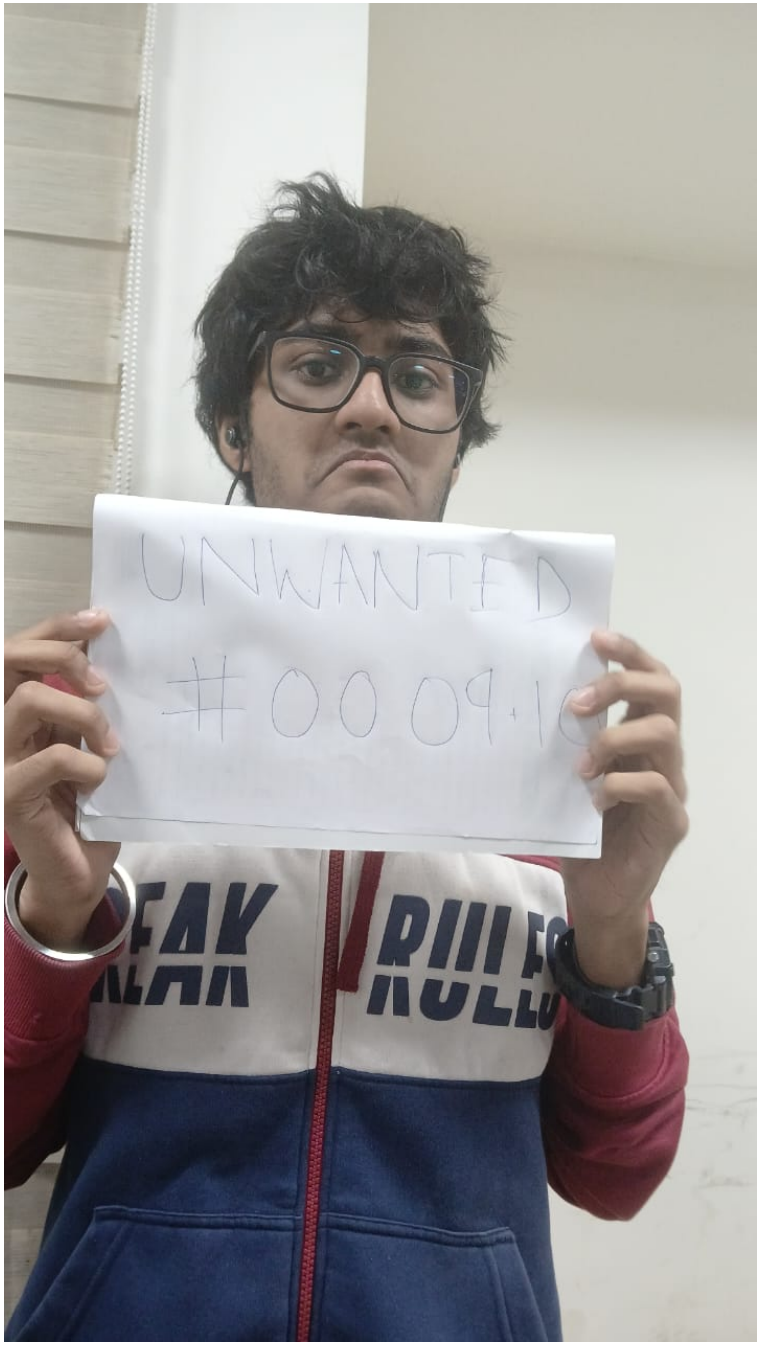


Figure: Local Search Heuristics Comparison

Conclusion

- Strong hybrid algorithm with mathematical reduction.
- Outperforms GA and Greedy significantly.
- Scales to very large instances using core reduction.

Future Work

- GPU parallelisation.
- Adaptive choice between Tabu and SA.
- Improved Lagrangian-based upper bounds.

Questions?